Atividade integradora

Alunos: Fernando Sousa Queiroz, Felipe Da Silva Oliveira e Giselle Diogo Martins

1. Classe funciona como um molde, uma planta para a estrutura, enquanto o objeto é uma instância criada a partir desse modelo. Ex.: A classe é a receita do bolo enquanto o objeto é o próprio bolo já feito.
2. Atributos são as próprias características do objeto e métodos são as ações que o objeto pode fazer. O estado é o que um objeto é (características, propriedades ou atributos) enquanto o comportamento é o que um objeto faz (suas ações)
3. New você cria uma nova instância. Quando você cria um objeto a instância que ocupa o espaço na memoria e a referencia é como você se refere a aquele objeto. Aluno aluno1 = new aluno()
4. É o ato de você proteger os atributos de uma classe. Para limitar o acesso dos usuários aos dados do objeto que não devem ser facilmente modificados, como por exemplo o extrato de uma conta bancaria deve ser feita em private.
5. Inicializar um objeto no momento exato em que foi criado. This é usado para referenciar um atributo da própria instância.
6. Agregação: É uma associação onde a classe agrupa outra, mas as partes podem existir sozinhas. Ex.: Departamento e Professor, podem ter vários professores em um departamento, mas ele for extinto os professores continuam vivos.
   Composição: É uma forma de agregação onde o objeto todo é responsável por criar e destruir as partes. Ex.: Casa e Cômodo, se a casa for destruída os cômodos também serão.
   Associação: Uma classe usa a outra e interage com ela, mas não há uma hierarquia. Pessoa e Carro, uma pessoa pode dirigir um carro, mas a pessoa existe sem ele assim como o Carro também existe sem ela.
7. Arraylist é uma coleção que pode alterar o seu tamanho dinamicamente, é melhor usada do que um vetor quando você não tem certeza do tamanho máximo que pode precisar.
8. Superclasse é a classe pai, Classe animal e Classe gato a superclasse seria a Classe Animal enquanto a, “Classe filho”, gato, seria a Subclasse. Extends serve para criar heranças entre classes, enquanto Super é usado para referenciar e acessar membros da Superclasse.
9. Sobrecarga é quanto você cria 2 métodos com o mesmo nome com atributos diferentes e sobrescrita é quando você reescreve o comportamento de um método da classe pai na classe filho.
10. Polimorfismo permite que objetos de diferentes subclasses sejam tratados de forma semelhante, mesmo tendo comportamentos diferentes. Pois ela (Superclasse) representa um tipo mais geral, enquanto a subclasse é um tipo mais especifico.
11. Interface é o contrato que define comportamentos. Extends serve para criar heranças entre classes e o implements serve para adotar e cumprir o contrato de uma ou mais interfaces.

12. Herança e composição são duas formas de reutilizar o código e estruturar classes. A composição é melhor usada quando você tem uma relação do tipo “tem um”. Ex.: O autor tem um livro. Já herança é do tipo “é um”. Ex.: O gato é um animal.
13. Baixo acoplamento é quando as diferentes classes do sistema possuem o mínimo de dependência entre si, se integrando por meio de interfaces. A injeção pode ajudar por meio de Inversão de controle, Dependência de abstrações, facilidade de testes (Mocking) e Imutabilidade e transparência.
14. SRP: Uma classe deve ter apenas uma função, você não pode ter o método adotar na classe gato.
    OCP: As classes devem ser fechadas para modificação e abertas para extensão, criar uma classe Desconto com o método calcular().
    DIP: Você não deve depender dos módulos, você deve construir uma camada de abstração entre a dependência e o seu programa. O PedidoService depende de uma interface abstrata (ex: RepositorioDePedido ou ServicoDeNotificacao)
15. É um código pronto feito para resolver um problema recorrente em códigos. Tem alguns motivos como evitar a complexidade desnecessária do código.
16. Singleton é um padrão de projeto que serve para garantir que uma classe tenha apenas uma única instância em toda a aplicação, fornecendo ao mesmo tempo um ponto de acesso global a instância. A finalidade do construtor privado é impedir a instanciação direta da classe por outras.
17. Factory method é um padrão de projeto que define uma interface para criar um objeto de uma superclasse, mas deixa as subclasses decidir qual classe concreta instanciar.
18. Strategy organiza os descontos por categorias: comuns (aplicados normalmente), estudantis (para estudantes mediante comprovação) e promocionais (ofertas temporárias para atrair clientes ou aumentar as vendas).
19. O Observer notifica automaticamente os setores de estoque, financeiro e auditoria quando um pedido é confirmado, permitindo que cada um execute sua função: atualizar produtos, registrar o pagamento e armazenar o histórico da operação.
20. Adapter: ele adapta interfaces incompatíveis para que sistemas diferentes possam trabalhar juntos. Exemplo: integrar um sistema de pagamento que possui uma interface diferente da esperada pelo projeto. Facade: simplifica o acesso a um sistema complexo, reunindo várias operações em uma interface única. Exemplo: criar um método `finalizarCompra()` que coordena pagamento, estoque e emissão de nota fiscal.